

DiscoveryHub

System Architecture & Service Breakdown — Internship Training Project (Smarsh)

This document describes how the DiscoveryHub platform is split across four independently deployable backend services and one frontend, built by a team of four. It covers the architecture diagram, the tech stack used per service, why each service is designed the way it is, and how Kafka connects everything together.

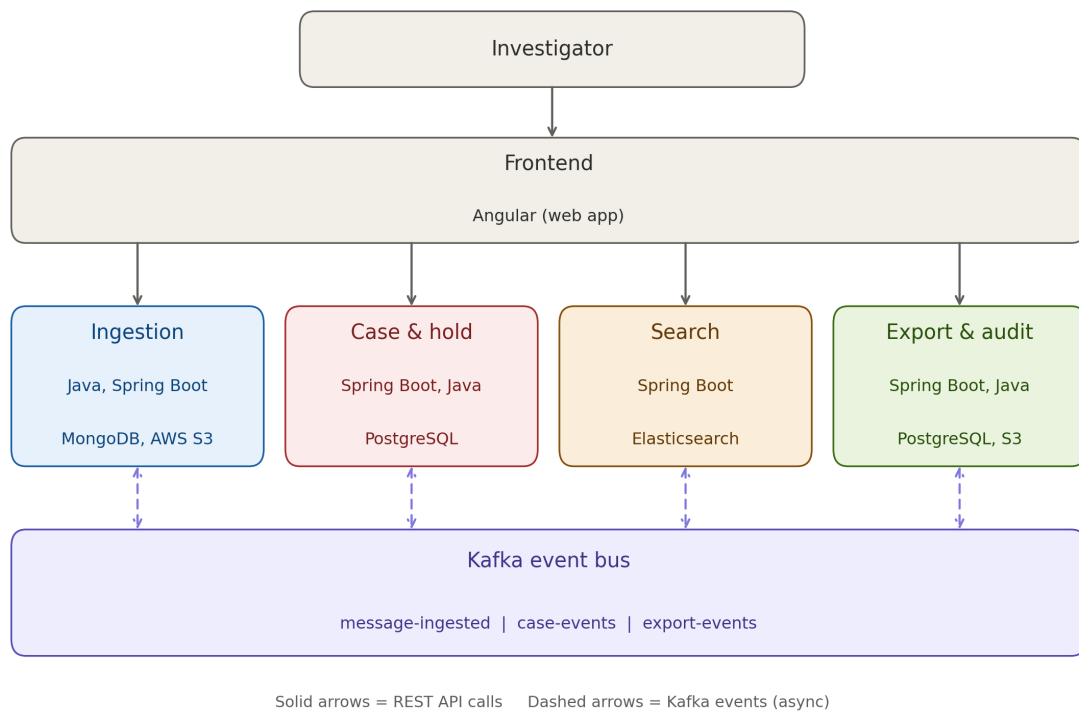


Figure 1: DiscoveryHub architecture — frontend, four backend services, and the Kafka event bus connecting them.

1. Overview

DiscoveryHub is a platform that lets an Investigator / Compliance Officer search business communications, place legal holds so messages can't be deleted, export evidence with verifiable checksums, and review a tamper-proof audit trail. There is a single user persona and no login — the app opens directly to the working screens.

With four team members, the system is split into four independently deployable backend services (satisfying the minimum-four-services requirement) plus one Angular frontend. Each service owns its own database — no shared schemas — and all inter-service communication happens either through direct REST calls from the frontend, or asynchronously through Kafka.

2. Services, tech stack, and reasoning

2.1 Person 1 — Ingestion & Archive Service

What it does: The 'front door' of the system. Generates the fake corpus (10,000+ messages), accepts incoming emails/chats, and saves them permanently with a unique ID and their attachments.

Core Java (multithreading)	Generating and processing 10,000+ messages one-by-one would be slow. Multiple threads let the service create/ingest many messages at once, like having several workers instead of one.
Spring Boot	Provides the API that 'accepts' a message, and separate background listeners that 'store' it — keeping accept and store as two separate steps (required by FR-1.4).
Kafka	Publishes every accepted message to a message-ingested topic instead of calling other services directly. This is what makes the accept/store separation real.
MongoDB	Messages don't all look the same (email vs chat, varied attachments), so a flexible, schema-less database is easier than rigid table columns.
AWS (S3)	Attachment files (PDFs, images, etc.) are best kept in file storage, not the database itself.
Design pattern	Factory Pattern — creates 'Email' or 'Chat' message objects cleanly depending on type.

Why this is the best fit: It's the first stage of the pipeline, so it makes sense as its own standalone service that other services simply 'listen to' via Kafka — nobody else needs to touch how messages come in.

2.2 Person 2 — Case Management & Legal Hold Service

What it does: Lets an investigator create a case, attach custodians, move the case through its stages (Draft → Active → Under Review → Closed), and place a 'hold' that legally protects messages from deletion.

PostgreSQL	Cases, custodians, and holds have clear structure and strict rules (e.g. a case can't skip a stage). A relational database is built for exactly this kind of 'must follow the rules' data.
-------------------	--

Spring Boot	Handles the REST APIs for creating cases/holds and validating stage transitions.
Kafka	Consumes message-ingested so a hold's scope automatically catches newly-arriving messages; publishes to case-events whenever a hold is placed, released, or a case changes stage.
Core Java (multithreading)	Placing a hold over thousands of messages must happen in the background without freezing the app (FR-4.3); multithreading speeds up marking messages once the event is picked up.
Design patterns	State Pattern for the case lifecycle (only certain transitions allowed); Strategy Pattern for matching different hold scopes (by custodian, date, or keyword).

Why this is the best fit: A hold only ever exists inside a case, so keeping them together avoids two services constantly talking to each other for something that's really one workflow.

2.3 Person 3 — Search Service

What it does: Powers full-text search across all messages with filters, highlighted keywords, sorting, and pagination — must return results in under 2 seconds even with 10,000+ messages.

Elasticsearch	The tool built specifically for fast full-text search with highlighting and filtering — a normal database would be too slow for this at scale.
Spring Boot	Exposes the search API that the frontend calls.
Kafka	Consumes message-ingested in the background and indexes each new message into Elasticsearch. This is why messages become searchable within 30 seconds (FR-1.7) without Search ever polling Ingestion.

Why this is the best fit: Search has very different performance needs than the rest of the system, so isolating it means a slow or heavy search index never affects case management or exports.

2.4 Person 4 — Export & Audit Service

What it does: Packages evidence into a downloadable, checksum-verified export, and separately keeps a permanent, tamper-proof log of every action anyone takes in the system.

PostgreSQL	Used for the audit log, with database rules that prevent any UPDATE or DELETE (truly append-only, satisfying FR-7.3), and for tracking export job status (Queued/Running/Completed).
Kafka	Consumes message-ingested, case-events, and export-events (which this service also publishes) — everything happening in the system flows into this one listener, which writes each event as an audit entry. This is the Observer Pattern in action.
AWS (S3 + presigned URLs)	Stores the finished export package and generates a download link that expires automatically.

Core Java

Computing checksums for every file and the whole package (so tampering can be detected), and streaming/zippping large exports without running out of memory.

Why this is the best fit: Both export and audit are about producing trustworthy, unchangeable records — pairing them means one person owns the 'proof and evidence' side of the system end-to-end, and Kafka lets that person capture everything happening across all four services without custom integration code for each one.

3. Kafka topics summary

Topic	Published by	Consumed by
message-ingested	Ingestion service	Search, Case & Hold, Export & Audit
case-events	Case & Hold service	Export & Audit
export-events	Export & Audit service	Export & Audit (own status tracking)

4. How the services fit together

- **Separate data, no shared schemas:** Ingestion uses MongoDB, Case/Hold uses PostgreSQL (its own schema), Search uses Elasticsearch, Export/Audit uses a different PostgreSQL schema + S3. Services never reach into each other's databases — they only talk through Kafka topics or direct APIs where truly needed.
- **If one service goes down (NFR-2):** Kafka keeps events sitting on the topic even if a consumer is offline. If Search crashes, ingestion keeps publishing to message-ingested normally; once Search restarts, it catches up and nothing is lost. This is the live resiliency demo: stop a service, keep ingesting, restart it, show it catch up.
- **Frontend (Angular):** Each person builds the Angular pages for their own service (e.g. Person 3 builds the search page), then the team assembles them into one app with a shared dashboard/home page (FR-8.2).
- **Deployment:** Each service gets its own Docker container, plus one Kafka container (KRaft mode avoids a separate Zookeeper container). A single docker-compose up starts everything together (NFR-4). Kubernetes is only a stretch goal.
- **Design pattern tie-in:** Kafka usage across the whole system is essentially the Observer / Publish-Subscribe pattern implemented at the architecture level — worth calling out explicitly since it directly satisfies the design-patterns requirement.